CMSO + atomRDF Data Flow: From Atomic Structure to Semantic Query

Layer 3: TBox (OWL Ontology — 5 Modules, OCDO Suite)
CMSO
AtomicScaleSample, Material, CrystalStructure, SpaceGroup, UnitCell, SimulationCell, Atom
hasCrystallographicDefect
hasCalculatedProperty hasComputationalMethod
hasCrystallographicDefect
anchors elements
anchors lattice
PODO
Vacancy, Substitutional, Interstitial
ASMO
DFT, EAM, kMC, FormationEnergy, BulkModulus, EnergyCutoff, KPointMesh
PLDO
GrainBoundary, StackingFault, Dislocation
ChEBI (chemical elements)
Wikidata (Bravais lattice)
defines classes & properties
defect types
method & property vocabulary
hasUnit
hasDislocation
QUDT (units: eV, Å³)
LDO
LineDefect, BurgersVector, SlipSystem, CharacterAngle
Layer 1: Input Sources (Atomic Parameters / Simulation Output)
Materials Project (API JSON)
LAMMPS / VASP (output files)
pyiron (job objects)
Crystal Parameters Fe/bcc, Cu/fcc, Al/fcc, Si/diamond
Defect Specs vacancy, substitutional, interstitial
API key
output path
element, crystal structure, a
type, count, repeat
Layer 2: atomRDF Python API (Structure → RDF)
WorkflowParser query_mp() parse_job()
atomrdf.build.bulk() ase + spglib → auto RDF
atomrdf.build.defect .vacancy() / .substitutional() .interstitial() / .dislocation()
kg.graph.add() Manual ASMO triples (method + property)
Auto-bridging: Python objects → ase.Atoms → pyscal3 analysis → RDF triples
parsed triples
75 triples / sample
+ defect triples
ASMO triples
Layer 4: ABox (RDF Instance Data — KnowledgeGraph)
KnowledgeGraph (rdflib.Graph)
~75 triples per bulk sample
~79 triples per defect sample
serialize(format="turtle")
samples.ttl / full_kg.ttl (Turtle serialization)
kg.visualise()
kg.query(ontology.terms...)
kg.query(sparql_string)
fix property names & namespaces
Layer 5: Query & Visualization
kg.visualise() Graphviz SVG
tools4RDF Declarative query (auto path-finding)
SPARQL Queries kg.query() 4 CQ + custom cross-ontology

⚠ Documentation–Implementation Drift

1. hasAltName: OWL defines it, atomRDF never emits it
2. hasVolume: returns node, not literal (need asmo:hasValue)
3. bulk() returns ase.Atoms, not (Atoms, Sample) tuple
4. Namespace: cdos/cdco/ & cdos/podo/ vs cmso/ & podo/

→ SPARQL must match ABox output, not OWL spec

Query Results

CQ1: SpaceGroup per sample
CQ2: bcc/fcc filter (drift!)
CQ3: Volume range (drift!)
CQ4: Atom count filter